

OSS Pulse: How Coding Agents Reshaped Open-Source Development

Tracking 1.46 Billion GitHub Events Across the 2022–2026 Agent Era

Jhe Chen Li (jl17797) Bo Yu (by2566)
Group 9 — BDAD Spring 2026, New York University

May 2026

Abstract

We present **OSS Pulse**, a large-scale empirical study of open-source software ecosystem health across the 2022–2026 coding agent era. By combining 1.46 billion GitHub Archive events (five quarterly snapshots), 2.8 million Hugging Face Hub model records, and PyPI download statistics for 46 AI/ML libraries, we construct a multi-dimensional health framework that goes beyond traditional star-based metrics. Our analysis reveals three fundamental shifts: (1) development became *lighter*—51% more developers but 42% fewer contributors per repository; (2) the weekend activity gap disappeared as automated agents operate around the clock; and (3) agent adoption followed a two-phase pattern—explosive growth in 2025 followed by normalization in 2026. We processed 930+ GB of raw data into 87.9 GB of cleaned Parquet files through 10 Spark analytics jobs on a Google Cloud Dataproc cluster.

1 Introduction

GitHub stars have long served as a proxy for open-source project health. However, in the era of coding agents—tools such as Cursor, Claude Code, and Devin that automate large portions of the software development workflow—this proxy is breaking down. DeepSeek-R1, for instance, ranks second on GitHub by stars with over 93,000 watchers, yet only 5 people actually push code to the repository.

The period from 2022 to 2026 spans a remarkable transformation in software development: from the pre-ChatGPT baseline (2022), through the LLM explosion (2023–2024), into the agent era (2025–2026). We hypothesize that coding agents are fundamen-

tally altering open-source contribution patterns, and that traditional metrics fail to capture these changes.

Research questions.

1. How has the macro-level structure of the OSS ecosystem changed across five annual snapshots?
2. Do coding agents produce measurable changes in development rhythms?
3. Can a multi-dimensional health score better characterize project vitality than star counts alone?

Contributions.

- A longitudinal dataset of 1.46B GitHub events across five Q1 snapshots (2022–2026).
- A composite health score integrating Hugging Face adoption, PyPI downloads, and GitHub activity.
- Empirical evidence of three structural shifts in OSS development driven by coding agents.
- An open-source Spark pipeline of 10 analytics jobs.

2 Related Work

OSS health metrics. Prior work has studied project health through contributor diversity [2], bus factor analysis [1], and community engagement metrics. The CHAOSS project defines standardized community health indicators, but most studies predate the agent era.

GitHub star inflation. Daoudi et al. [3] identified approximately 6 million fake stars on GitHub, with 90% of flagged repositories subsequently deleted. Our 2026 data—showing a 63% drop in WatchEvents—aligns with this large-scale cleanup.

Table 1: GitHub Archive data scale per era.

Era	Events	Repos	Actors
2022-Q1	206.8M	17.0M	7.0M
2023-Q1	247.4M	19.9M	7.8M
2024-Q1	364.4M	17.6M	8.9M
2025-Q1	376.4M	20.2M	10.2M
2026-Q1	265.9M	20.9M	10.6M
Total	1.46B	—	—

AI-assisted development. Recent surveys document the rise of coding agents [4], but empirical, large-scale analyses of their ecosystem-wide effects remain scarce. Our work fills this gap by quantifying behavioral shifts across 1.46B events.

3 Data Sources

We integrate three complementary data sources to enable multi-dimensional health assessment.

3.1 GitHub Archive

GH Archive¹ provides hourly snapshots of all public GitHub events. We collected five Q1 windows (January–March) for 2022 through 2026, plus the full 2025 calendar year as supplementary data. Table 1 summarizes the scale. Five event types are retained: `PushEvent`, `PullRequestEvent`, `WatchEvent`, `IssuesEvent`, and `ForkEvent`.

3.2 Hugging Face Hub

We captured a snapshot of the Hugging Face Hub in April 2026, comprising 2,815,064 model records. Key fields include `modelId`, `pipeline_tag`, `library_name`, `downloads` (30-day), `likes`, and `parameter_count`. The data was ingested via the `huggingface_hub` Python API and processed through a Scala Spark pipeline.

3.3 PyPI Download Statistics

Monthly download counts for 46 AI/ML Python libraries were obtained via Google BigQuery’s public PyPI dataset, covering all 12 months of 2025.

¹<https://www.ggharchive.org/>

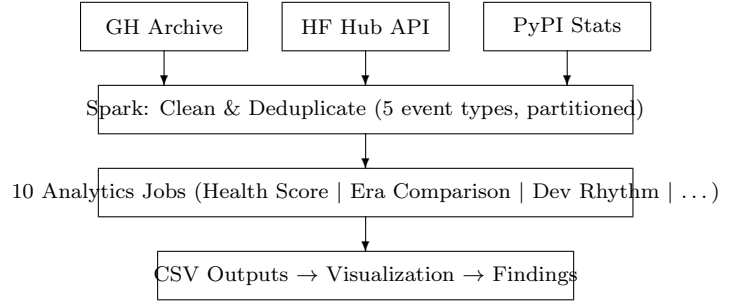


Figure 1: Pipeline architecture: three data sources feed into Spark cleaning, then fan out to 10 analytics jobs.

3.4 Seed Repository List

We curated 93 AI/ML repositories spanning 14 categories (frameworks, LLM serving, inference, agents, RAG, diffusion, training, evaluation, etc.) to enable focused AI-vs-general comparisons.

4 Pipeline Architecture

4.1 System Overview

The pipeline runs on NYU’s Google Cloud Dataproc cluster (4 worker nodes, n1-standard-16; master n1-standard-32; 219 GB YARN memory, 64 vCores). All data is stored in HDFS as Parquet files partitioned by `event_date`.

The architecture follows a three-stage design (Figure 1):

- Ingest:** Raw JSON.gz files are parsed with explicit schemas, converted to Parquet with column pruning (70% storage reduction).
- Clean:** Deduplication by `event_id`, null filtering, timestamp normalization, and schema unification.
- Analyze:** 10 independent Spark jobs produce CSV outputs for downstream visualization.

Total data reduction: 930+ GB raw → 87.9 GB cleaned (90.5%).

4.2 Schema Design

All five event types are mapped to a unified schema with 10 fields: `event_id`, `event_type`, `event_date` (partition key), `event_ts`, `actor_login`, `repo_name`,

Table 2: Spark analytics jobs.

#	Output	Description
01	repo.daily	Daily activity per AI repo
02	hf_gh_join	HF models + GH stats
03	health_score	Composite health score
04	top_repos	Top 1000 repos (ecosystem)
05	ai_vs_general	AI vs. general repos
06	star_hype	Star growth & hype
07	contributor	Bus factor, concentration
08	era_comparison	Cross-era (5 Q1 snapshots)
09	era_deep_dive	Per-repo era analysis
10	dev_rhythm	Weekend/weekday, push freq.

payload.action, pr.number, pr.merged, and push.distinct.size.

4.3 Analytics Jobs

Table 2 summarizes the 10 analytics jobs.

5 Data Quality

Across all five eras, the four core fields (event_type, event_date, actor_login, repo_name) have zero nulls. Payload-specific fields (pr.merged, pr.number, payload.action) are structurally null for non-applicable event types.

For the 2026-Q1 era, we re-downloaded all 2,160 raw hourly files and re-cleaned them. 580 corrupt gzip files were handled via Spark’s ignoreCorruptFiles option. Post-cleaning diff against the original was less than 0.39%. All eras are deduplicated by event_id.

6 Engineering Challenges

6.1 HDFS Quota Management

Processing five years of Q1 data required ~700 GB of raw storage, but the HDFS quota was 500 GB.

Solution 1—Rolling pipeline: Download one era’s raw data, clean to Parquet, delete raw, proceed to next era. Storage stays under quota.

Solution 2—Per-era processing: Compute metrics for each era independently, save intermediate results. A final merge combines only small inter-

Table 3: GitHub API schema change.

Field	≤2025	2026+
pr.merged	true/false	NULL
action	open/close	+ “merged”
distinct.size	present	removed

Table 4: Ecosystem evolution: 2022 vs. 2026 Q1.

Metric	2022	2026	Δ
Developers	7.0M	10.6M	+51%
Total events	207M	266M	+29%
1-commit push %	87%	94%	+7pp
PR/push ratio	0.098	0.077	−21%
Contrib./repo	4.88	2.84	−42%

mediate CSVs. If YARN terminates mid-job, completed eras are preserved.

6.2 2026 Schema Breaking Change

Job 08 initially reported zero merged PRs for 2026-Q1. Investigation revealed that GitHub changed its API payload schema in late 2025: the pr.merged field became permanently NULL, replaced by a new payload.action=“merged” value (Table 3).

We implemented a backward-compatible fix:

```
merged = (
    ((col("payload_action") == "closed")
     & col("pr_merged").eqNullSafe(True))
    | (col("payload_action") == "merged")
)
```

6.3 Billion-Record Shuffle Optimization

Computing top-1000 repos across 1B+ events required a full cross-partition shuffle (72 GB → 83 GB shuffle, >3 hours). We tuned shuffle.partitions (400→200), cached DataFrames, and downstream jobs read pre-aggregated outputs. **Retrospective:** Bucketing by repo_name at ingest would eliminate the shuffle entirely.

7 Findings

7.1 Finding 1: Five-Year Ecosystem Shift

Figure 2 visualizes the ecosystem evolution; Table 4 presents the key metrics.

More developers are participating, but each repo receives fewer contributors. Pushes are smaller and

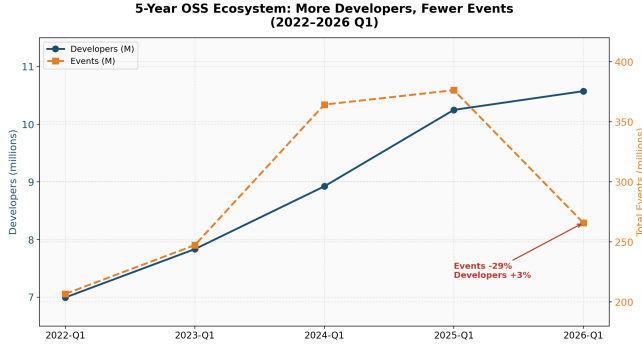


Figure 2: Five-year ecosystem shift across 2022–2026 Q1 snapshots. Developers grew 51% while contributors per repo fell 42%.

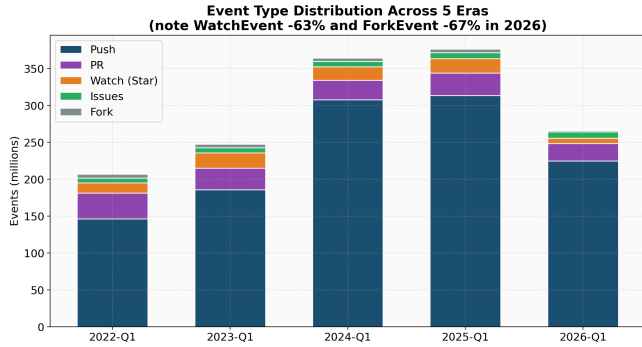


Figure 3: Event type composition across five eras. Push dominance grows as popularity signals (Watch, Fork) decline sharply in 2026.

more frequent, while the PR/push ratio declined 21%—less ceremony, more direct pushes.

The 2026 event drop. Total events declined 29% from 2025 to 2026. This drop is concentrated in popularity signals: WatchEvents –63%, ForkEvents –67%, consistent with GitHub’s removal of ~6M fake star accounts [3]. PushEvent’s share grew from 71% to 85%—engineering activity was least affected.

7.2 Finding 2: The Weekend Gap Disappeared

In a natural weekly distribution, weekends account for 28.9% of days (2/7). Human developers historically produce a weekend gap. Figure 4 shows this gap vanishing by 2026.

Push-specific weekend ratio: 24.8% (2022) → 29.4% (2026). By 2026, GitHub activity shows no weekly periodicity—consistent with agents operating around the clock.

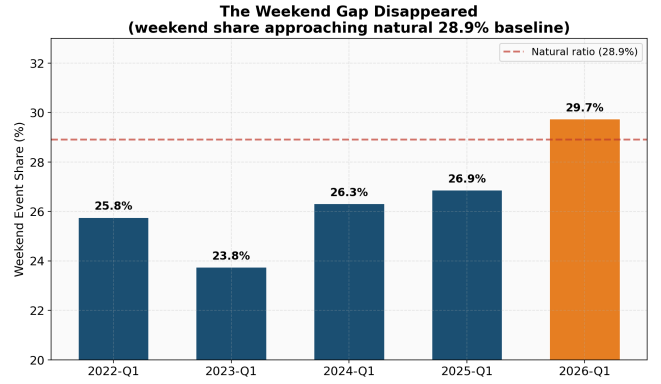


Figure 4: Weekend event share across five eras. By 2026, weekend activity exceeds the 28.9% natural baseline—agents work 24/7.

Table 5: Weekend event share across five eras.

Era	Wknd %	vs. 28.9%
2022-Q1	25.8%	–3.1pp
2023-Q1	23.7%	–5.2pp
2024-Q1	26.3%	–2.6pp
2025-Q1	26.9%	–2.0pp
2026-Q1	29.7%	+0.8pp

7.3 Finding 3: Two Phases of Agent Adoption

The agent era exhibits a distinctive two-phase pattern (Figure 6, Table 6).

Phase 1—Explosion (2025): High-frequency push accounts nearly doubled (>1K pushes: 5,347→9,788). Early adopters ran coding agents at superhuman rates.

Phase 2—Normalization (2026): High-frequency accounts dropped –35% as GitHub cleaned bots, while total push actors grew to 8.71M (+25%). Average pushes/actor fell 43%. The signature: *more people push, but each pushes less*.

7.4 Finding 4: Stars ≠ Health

Our composite health score reveals that star counts are poor predictors of project health:

$$H = 0.3S_{HF} + 0.2S_{PyPI} + 0.15S_{\star} + 0.15S_{push} + 0.1S_{PR} + 0.1S_{days} \quad (1)$$

where each S is a min–max normalized score.

Figure 7 and Table 7 illustrate how three distinct archetypes emerge.

Sentence-transformers has 1,588 stars but 519M HF downloads—invisible to star-based rankings. Ol-

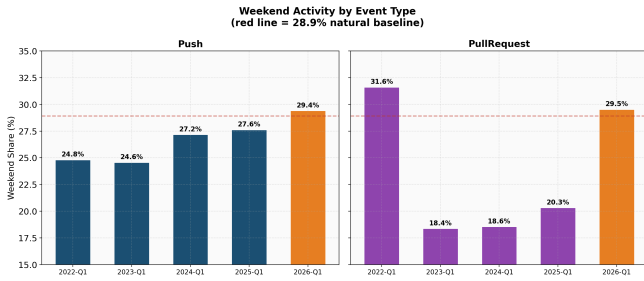


Figure 5: Weekend activity ratio by event type. Push weekend ratio converges to the natural baseline by 2026.

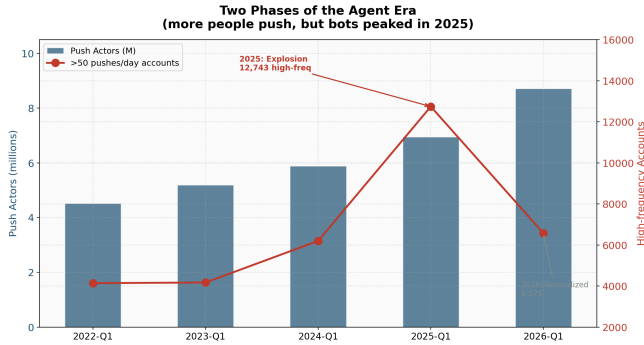


Figure 6: Two phases of agent adoption: 2025 explosion (high-frequency pushers tripled) followed by 2026 normalization.

lama has 44.5K stars but only 241 HF downloads, because it uses its own model registry. No single metric suffices; triangulation is essential.

7.5 Finding 5: Contributor Concentration

AI repos exhibit systematically higher contributor concentration (Figure 8).

Key examples:

- **open-webui**: `top1_push_ratio` = 0.997 (one person, 99.7% of pushes), yet 957 PR contributors.
- **DeepSeek-R1**: 93,947 watchers, 5 push contributors.
- **pytorch**: `top1_push_ratio` = 0.14, 1,605 PR contributors—healthy distribution.

A caveat: TensorFlow’s top pusher is a CI bot. Push concentration and PR diversity must be examined together.

Table 6: Push actor statistics (2024–2026).

Metric	'24	'25	'26
Push actors	5.89M	6.95M	8.71M
>1K pushes	5,347	9,788	6,351
>50/day	6,197	12,743	6,575
Avg push/actor	52.2	45.1	25.8
Median pushes	6	6	4

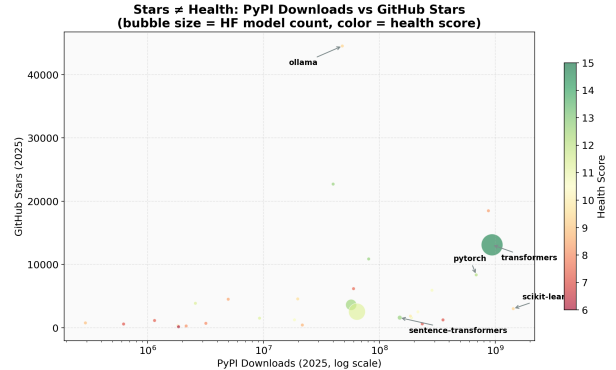


Figure 7: Stars vs. health score for AI repositories. High stars do not guarantee high health (e.g., Ollama).

8 Discussion

Implications for OSS health measurement.

The “GitHub stars” metric is becoming actively misleading in the agent era. As agents generate automated activity, the signal-to-noise ratio of any single metric degrades. Multi-dimensional frameworks—combining community engagement (HF downloads), demand signals (PyPI installs), and engineering intensity (PRs, push diversity)—are necessary.

The “lighter development” paradox. More developers than ever (10.6M, +51%), yet per-repo contributor counts dropped 42%. Agents may enable individuals to contribute to more projects simultaneously, but with shallower engagement.

24/7 development and sustainability. The vanished weekend gap raises sustainability concerns. While partly attributable to automated agents, it may also reflect increasing pressure on human developers. Distinguishing human from agent activity is critical for future work.

Limitations.

- Q1-only sampling may miss seasonal effects.
- We cannot reliably distinguish agent-generated from human-generated events at the individual level.

Table 7: Three health archetypes.

Archetype	Example	Stars	Usage Signal
True leader	transformers	13K	PyPI 938M, HF 835K
Quiet power	sent.-trans.	1.6K	HF 519M downloads
Eco. outsider	ollama	44.5K	PyPI 47.8M, HF 241

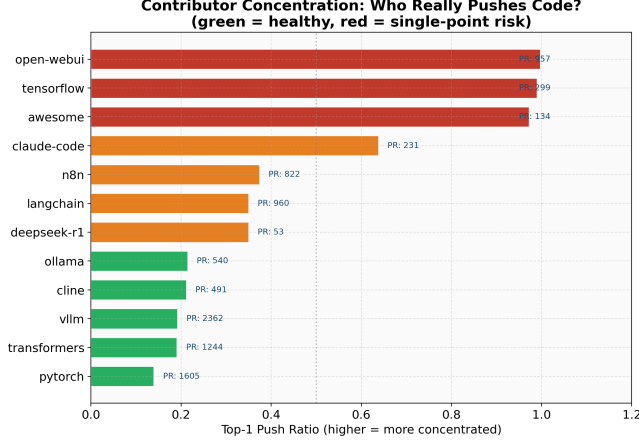


Figure 8: Contributor concentration across top repositories. Red indicates high bus-factor risk.

- The 2026 event drop conflates GitHub’s account cleanup with genuine behavioral changes.
- Health score weights are heuristic; sensitivity analysis would strengthen results.
- The 93 seed repos may not fully represent the AI ecosystem.

9 Lessons Learned

1. **Design for the constraint.** The 500 GB quota forced a rolling pipeline that proved more crash-safe than a monolithic approach.
2. **Never trust a live API schema.** The 2026 change silently broke our merge query—always assert assumptions.
3. **Validate anomalies externally.** The 2026 event drop appeared to be a bug; cross-referencing with [3] confirmed it was real.

10 Conclusion

We analyzed 1.46 billion GitHub events across the 2022–2026 coding agent era, complemented by Hugging Face and PyPI data. Three key findings:

1. **Development got lighter:** +51% developers, −42% contributors per repo, smaller and more

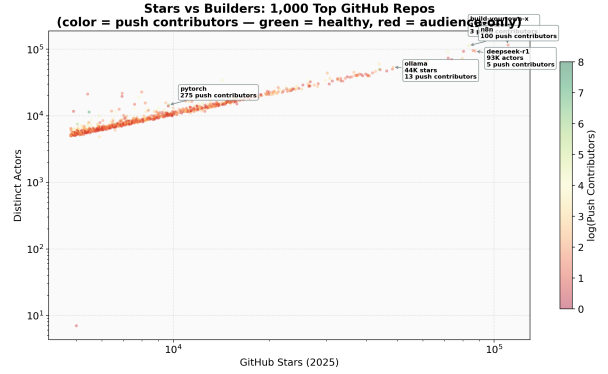


Figure 9: Stars vs. actual push contributors. DeepSeek-R1: 93K watchers, 5 pushers.

frequent pushes.

2. **Weekends disappeared:** Weekend activity reached 29.7%, erasing the human work–rest signal.
3. **Two-phase adoption:** 2025 explosion (automated pushers tripled) → 2026 normalization (broader but lighter).

Traditional metrics like stars are increasingly unreliable. Multi-dimensional measurement—community adoption, package downloads, and engineering activity—is essential for accurate OSS health assessment.

Future work includes: developing heuristics to distinguish agent from human events; extending to full-year coverage; incorporating code review latency and issue resolution time; and longitudinal tracking of individual project health trajectories.

Acknowledgments

We thank NYU High Performance Computing for providing the Google Cloud Dataproc cluster, GH Archive for open hourly event logs, Hugging Face Hub for public model metadata, and PyPI for download statistics.

References

- [1] Guilherme Avelino, Leonardo Passos, Andre Hora, and Marco Tulio Valente. A novel approach for estimating truck factors. In *Proceedings of the 24th International Conference on Program Comprehension (ICPC)*, pages 1–10. IEEE, 2016.

- [2] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 186–196. ACM, 2017.
- [3] Noureldin Daoudi, Filipe R. Cogo, Bram Adams, and Foutse Khomh. Unveiling fake stars: A study of star inflation in open-source software. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, pages 1–12, 2024. Found ~6 million fake stars on GitHub; 90% of flagged repos later deleted.
- [4] Angela Fan et al. Large language models for software engineering: Survey and open problems. *arXiv preprint arXiv:2310.03533*, 2023.